



Intro to Metalanguages

Programming Languages

Introduction

A metalanguage is a language used to describe another language.

Primary Example: BNF (Backus-Naur Form).

- BNF is the standard metalanguage used to define programming language syntax.
- It uses **abstractions** to represent syntactic structures.

Anatomy of a BNF rule

The Production (Rule): The entire definition is called a rule or "production."

- **Structure:**

$\langle LHS \rangle \rightarrow \langle RHS \rangle$

Anatomy of a BNF rule

Left-Hand Side (LHS):

- The abstraction being defined.
- Usually delimited by pointed brackets (e.g., <assign>).

Right-Hand Side (RHS):

- The definition of the LHS.
- Contains a mixture of tokens, lexemes, and references to other abstractions.

Analyzing a Rule (Example)

Defining a Java assignment statement.

The BNF Rule:

$$\langle \textit{assign} \rangle \rightarrow \langle \textit{var} \rangle = \langle \textit{expression} \rangle$$

<assign> is defined as:

1. An instance of <var>
2. Followed by the lexeme =
3. Followed by an instance of <expression>

Terminals vs. Nonterminals

Nonterminal Symbols (Nonterminals):

- The abstractions used in the grammar.

Ex. <assign>, <var>, <expression>.

Terminal Symbols (Terminals):

- The actual lexemes and tokens that appear in the code.

Ex. =, if, (,), +.

Grammar: A collection of these rules.

Handling Multiple Definitions

The Scenario: A nonterminal may have two or more distinct syntactic forms.

The Operator: The vertical bar | represents a logical **OR**.

Example: A Java if statement.

- *Option A:* if (<logic_expr>) <stmt>
- *Option B:* if (<logic_expr>) <stmt> else <stmt>

$$\langle if_stmt \rangle \rightarrow \text{if}(\langle logic_expr \rangle) \langle stmt \rangle \mid \text{if}(\langle logic_expr \rangle) \langle stmt \rangle \text{ else } \langle stmt \rangle$$



Lists in BNF

Programming Languages

Describing Lists in BNF

BNF does not have a specific iteration syntax.

The Solution: Use **recursion** to describe variable-length lists.

Example (Ident List):

- Rule: $\langle \text{ident_list} \rangle \rightarrow \text{identifier} \mid \text{identifier}, \langle \text{ident_list} \rangle$
- Interpretation: A list is a single identifier, OR an identifier followed by a comma and another list.

Grammars and Derivations

Derivation: The process of generating a specific sentence from a start symbol.

Key Concepts:

- **Sentential Form:** Every string of symbols in a derivation.
- **Leftmost Derivation:** Replacing the leftmost nonterminal in each step.
- **Rightmost Derivation:** Replacing the rightmost nonterminal in each step.

Ambiguity in Grammars

Definition: A grammar is **ambiguous** if it generates a sentential form that has:

- Two or more distinct parse trees.
- (Equivalently) Two or more distinct leftmost derivations.

Problem: Compilers cannot determine the correct structure (e.g., if-else dangling problem).

Resolution: Rewrite the grammar to be unambiguous (often by adding more nonterminals/levels).

Operator Precedence & Associativity

Precedence:

- Determines which operator is evaluated first (e.g., * before +).
- In BNF: Handled by placing operators with higher precedence "lower" in the parse tree (further from the start symbol).

Associativity:

- Determines order for operators of equal precedence (e.g., $A - B - C$ is $(A - B) - C$).
- **Left Associative:** Uses left recursion ($\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$).
- **Right Associative:** Uses right recursion ($\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle + \langle \text{expr} \rangle$).



Attribute Grammars

Programming Languages

Static Semantics & Attribute Grammars

Static Semantics: Rules that are checked at compile time but are difficult to describe with BNF (e.g., "variables must be declared before use").

Attribute Grammars (AGs):

- An extension of Context-Free Grammars (CFGs).
- Add **Attributes** (values) to grammar symbols.
- Add **Semantic Functions** to compute those values.
- Add **Predicate Functions** to check validity.

Attributes types

Synthesized Attributes:

- Pass information **up** the parse tree (from children to parents).

Inherited Attributes:

- Pass information **down** the parse tree (from parents/siblings to children).

Intrinsic Attributes:

- Values determined outside the parse tree (e.g., from the symbol table).

Examples

- **Syntax rule:** $\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$

Semantic rule: $\langle \text{expr} \rangle.\text{expected_type} \leftarrow \langle \text{var} \rangle.\text{actual_type}$

- **Syntax rule:** $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle[2] + \langle \text{var} \rangle[3]$

Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow$

if ($\langle \text{var} \rangle[2].\text{actual_type} = \text{int}$) and

($\langle \text{var} \rangle[3].\text{actual_type} = \text{int}$)

then int

else real

end if

Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$

Examples

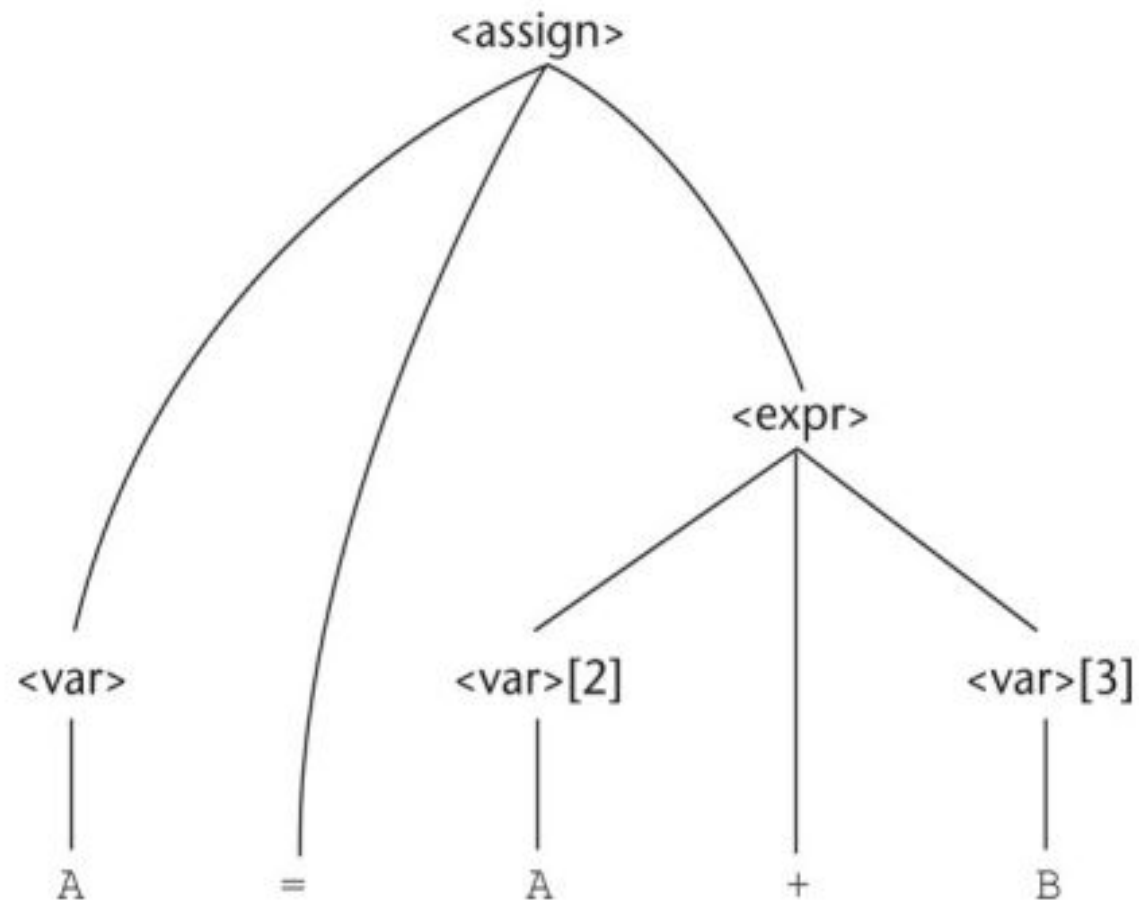
- **Syntax rule:** $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle$

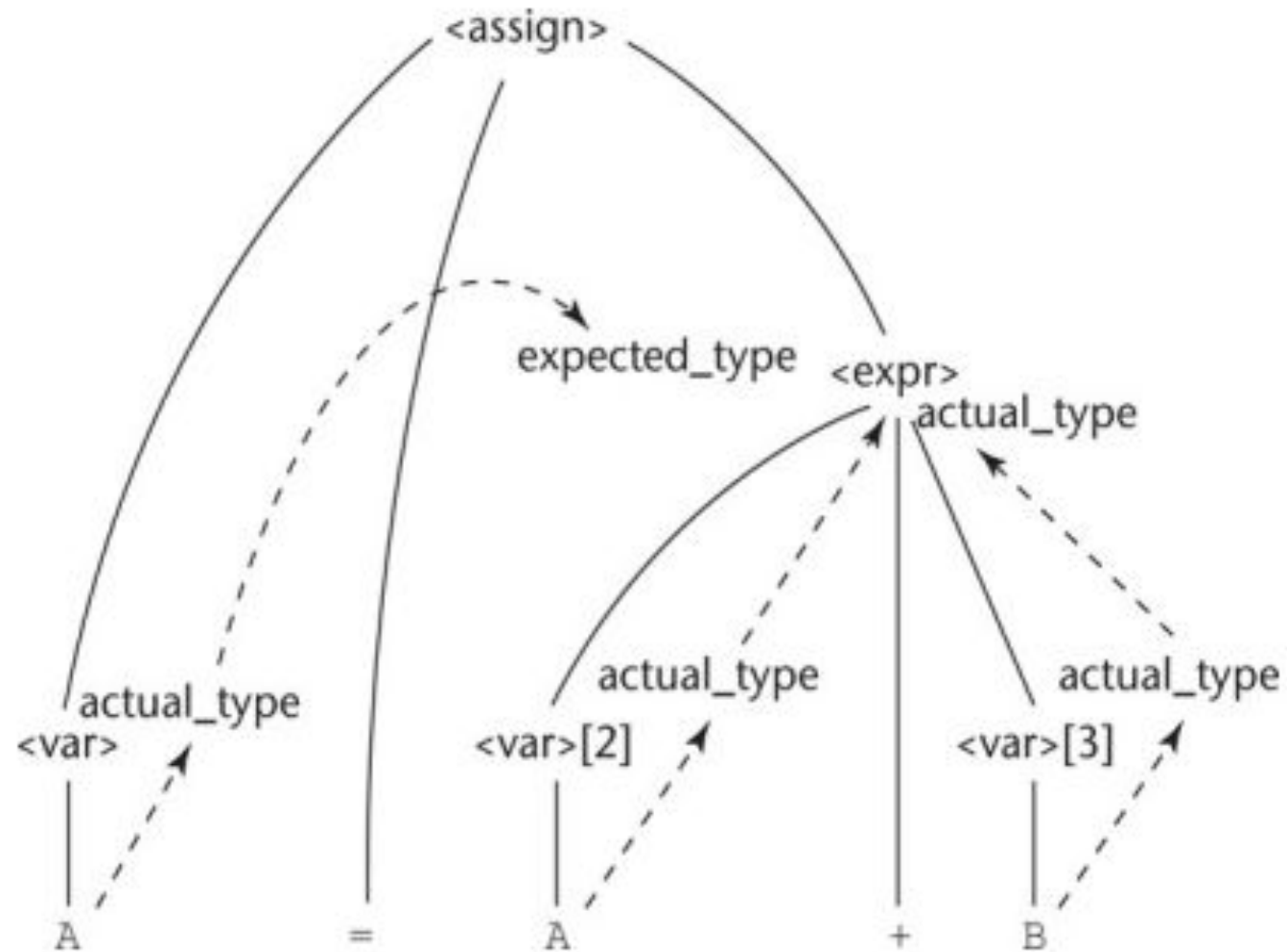
Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow \langle \text{var} \rangle.\text{actual_type}$

Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$

- **Syntax rule:** $\langle \text{var} \rangle \rightarrow A \mid B \mid C$

Semantic rule: $\langle \text{var} \rangle.\text{actual_type} \leftarrow \text{look-up}(\langle \text{var} \rangle.\text{string})$







Dynamic Semantics

Programming Languages

Introduction to Dynamic Semantics

Describing the *meaning* of expressions, statements, and program units (what happens when the code runs).

Why it's hard: No single universally accepted notation (unlike BNF for syntax).

Three Major Approaches:

- Operational Semantics
- Denotational Semantics
- Axiomatic Semantics

Operational Semantics

Describe the meaning of a statement by specifying the effects of running it on a machine.

Method:

- Define a virtual machine (intermediate language).
- Translation rules map source code to this machine code.

Ex. Describing a for loop using goto and conditional jumps.

Denotational Semantics

Mathematical approach based on recursive function theory.

Method:

- Define a mathematical object (the "denotation") for each language entity.
- Map syntactic entities to these mathematical objects.
- *Key*: The state of a program is essentially a mathematical function mapping variables to values.

Axiomatic Semantics

Based on mathematical logic; designed for **program verification** (proving correctness).

Method:

- **Preconditions:** What must be true *before* a statement executes.
- **Postconditions:** What must be true *after* a statement executes.
- **Weakest Precondition:** The least restrictive condition needed to guarantee the postcondition.